

Graph-Guided Reasoning and Refinement in Diffusion Language Models

Sriharini Margapuri (2026701030), Arkaprava Gain (2026701044),
Aadith Warriar (2025701048), Tarun R (2025701005)

CherrySoda

Abstract

Diffusion Language Models (DLMs) refine a masked sequence over many denoising steps, so in principle they can revise their own reasoning. However, existing correction mechanisms operate at the token or sequence level and do not explicitly represent dependencies between reasoning steps. We propose a training-free graph-guided framework that parses intermediate reasoning into a dependency graph and uses it to guide denoising and localized error correction, remasking only the reasoning subgraph affected by a detected inconsistency. Baseline experiments with LLaDA and Dream on GSM8K, SVAMP and ProofWriter show that most incorrect answers contain no detectable arithmetic slip, and that answer voting, token-dependency scheduling and an untrained token-level corrector do not repair them. We have built the rule-based graph parser and training-free step verifiers, which catch over 90% of injected arithmetic errors while flagging under 3% of correct steps. The remaining work integrates them into localized remasking and compares the framework with existing diffusion-based correction methods.

1 Introduction

Large Language Models (LLMs) have demonstrated strong performance on a wide range of reasoning tasks, particularly when prompted to generate intermediate reasoning steps (Wei et al., 2023). However, most conventional LLMs operate autoregressively, generating text sequentially from left to right. While effective, this sequential nature means that once a token is emitted, it cannot be revised. Consequently, an error made early in a chain of thought propagates through subsequent steps, corrupting the final answer. Techniques like self-consistency (Wang et al., 2023), step-level verification (Lightman et al., 2023), and self-refinement (Madaan et al., 2023) can mitigate these issues, but they often require regenerating

large portions of the reasoning process and fail to provide a straightforward mechanism for revising specific reasoning steps.

Diffusion Language Models (DLMs) have recently emerged as an alternative in natural language processing (NLP) (Austin et al., 2023; Sahoo et al., 2024). A masked DLM starts from a fully masked sequence and iteratively denoises it (Nie et al., 2025; Ye et al., 2025), and because any position can be re-masked and re-predicted, it can refine and correct its output during generation (Huang et al., 2025; Schiff et al., 2026; Yao, 2026). However, existing correction methods primarily operate at the token or sequence level and treat reasoning as unstructured text (Huang et al., 2025; Schiff et al., 2026; Yao, 2026; Wang et al., 2025). Without an explicit representation of how reasoning steps depend on each other, token-level correctors cannot determine which portions of the reasoning cause an error, so they often revisit large regions of otherwise valid reasoning, adding computational cost and potentially introducing new errors.

Our baseline experiments on two DLMs, LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025), suggest reasoning errors frequently involve dependencies between steps rather than isolated arithmetic mistakes. Across 5,838 generations from LLaDA and Dream, only 16–27% of incorrect answers on GSM8K (Cobbe et al., 2021) and SVAMP (Patel et al., 2021) contain an arithmetic mistake detectable through a local calculation check. In 58–75% of incorrect answers, every checkable calculation is correct, and the error lies elsewhere, most plausibly in how quantities are chosen and combined across steps. On ProofWriter (Tafjord et al., 2021), LLaDA’s accuracy on True and False questions falls from 95% to 60% as proofs grow from zero to three dependent steps. Furthermore, selecting among the answers a DLM produces during denoising (Wang et al., 2025) does not repair such errors, and applying a

token-level corrector (Schiff et al., 2026) to a model not trained for it reduces GSM8K accuracy from 80% to 9%. These findings suggest that improving reasoning requires more than revising individual tokens or selecting among candidate answers.

We argue that reasoning structure can play an active role in error correction. Many reasoning tasks naturally form dependency graphs where future conclusions rely on earlier steps, while others can be established independently. Such structure provides information not only about the order in which reasoning steps can be generated, but also about which steps must be reconsidered when an error is detected.

We propose a graph-guided diffusion framework that parses intermediate reasoning traces into a directed dependency graph and uses this graph for two purposes. First, it schedules dependency-aware denoising, allowing independent branches to be refined in parallel while dependent steps wait for their prerequisites to stabilize. Second, it localizes detected errors to the affected subgraph and remasks only the erroneous nodes and the nodes that depend on them. By regenerating only the affected reasoning, the framework aims to reduce unnecessary computation while preserving independently verified steps, without requiring additional training of the underlying DLM.

Our study is organized around four research questions.

- **RQ1.** Where do errors arise in multi-step reasoning with diffusion language models, and how many are locally detectable slips rather than structural errors in how steps are connected?
- **RQ2.** Does dependency-aware denoising improve reasoning accuracy and computational efficiency over existing decoding strategies at the same generation length, and how many forward passes does it need?
- **RQ3.** Can dependency-guided remasking improve accuracy by correcting only the steps affected by a detected error, compared with full-sequence remasking and with perfect localization?
- **RQ4.** How does the effectiveness of graph-guided diffusion vary across models, datasets, and reasoning complexity?

Our contributions are as follows:

- We introduce a step-level dependency graph for organizing diffusion-based reasoning. Unlike token-level dependency schedulers, the graph represents relationships between reasoning steps and uses their prerequisites to guide denoising.
- We develop training-free step verifiers and a dependency-guided correction mechanism that localizes a detected inconsistency to the affected subgraph and remasks only the erroneous nodes and their dependent steps. This enables localized regeneration while retaining unaffected reasoning.
- We establish a controlled evaluation of correction and scheduling baselines on two DLMs and three multi-step reasoning datasets: GSM8K, SVAMP, and ProofWriter. We compare answer voting, dependency-aware token scheduling (DAPD), and trained token-level correctors (RemeDi and ProSeCo), and analyze the types of errors that arise during diffusion-based reasoning.

2 Related Works

2.1 Reasoning in Diffusion Language Models

Diffusion language models generate text through iterative denoising rather than left to right (Austin et al., 2023; Sahoo et al., 2024). In masked diffusion, generation begins with masked positions that the model progressively predicts and commits to over multiple denoising steps. The order of commitment is not fixed, and committed tokens can in principle be masked again (Huang et al., 2025; Schiff et al., 2026; Yao, 2026). LLaDA employs confidence-guided, block-wise generation (Nie et al., 2025), while Dream initializes a diffusion model from a pretrained autoregressive (AR) model and supports arbitrary-order generation and infilling (Ye et al., 2025).

Diffusion-of-Thought (DoT) (Ye et al., 2024) extends Chain-of-Thought (CoT) reasoning to diffusion models by letting reasoning steps diffuse over time, so that parts of a trace can be revised before generation is complete, and reports promising self-correction abilities. d1 shows that supervised fine-tuning (SFT) and GRPO-style reinforcement learning (RL) improve reasoning in DLMs (Zhao et al., 2025). Yu et al. (2026) show that masked DLMs are far less sensitive than AR models to the order in which reasoning and answers are produced,

and attribute this to simpler reasoning tokens stabilizing before harder answer tokens.

However, the ability to revise tokens does not guarantee that an incorrect step is identified or corrected. Neither training nor iterative generation explicitly represents dependencies between steps, so a model may revise tokens without determining which steps caused an incorrect conclusion or must be reconsidered.

2.2 Self-Correction and Error Recovery in Diffusion Models

Iterative denoising lets generated content be revised before the sequence is complete, but a revision may keep an incorrect token, change a correct one, or introduce new errors. This motivates methods that make error detection and recovery more straightforward.

Several approaches address this problem by modifying the denoising process. RemeDi augments token prediction with confidence scores and uses them to identify tokens for remasking and resampling with additional context (Huang et al., 2025). Its correction mechanism is incorporated into training through SFT and RL. ProSeCo trains a correction model on errors arising from the denoiser’s own unmasking decisions and combines correction with generation (Schiff et al., 2026). Although both methods learn to correct errors, their interventions primarily operate at the token level rather than identifying incorrect reasoning steps and their downstream consequences.

Other methods investigate error recovery without training an additional model. Remask-Don’t-Replace revisits a suspect token by resetting it to [MASK] and allowing the denoiser to predict it again, rather than directly overwriting it (Yao, 2026). Temporal Self-Consistency Voting exploits the fact that a correct answer may appear at an intermediate denoising step and subsequently disappear (Wang et al., 2025). It selects an answer by aggregating predictions across denoising steps, without explicitly correcting the reasoning trace.

None of these methods represents dependencies among reasoning steps. An incorrect intermediate result may invalidate several downstream steps, while an unrelated error can be corrected on its own; token-level remasking does not separate these cases, and answer-level voting does not repair the reasoning behind an incorrect answer.

2.3 Dependency-Aware Decoding in Diffusion Models

Masked diffusion permits flexible generation orders, but predicting dependent tokens before their prerequisites are established can lead to inconsistent intermediate states (Kim et al., 2026; Ringel et al., 2026), which has motivated dependency-aware denoising.

Several approaches have modelled dependencies at the token level. DAPD estimates token dependencies and uses them to identify positions that can be denoised in parallel (Kim et al., 2026). DOS uses attention matrices to approximate inter-token dependencies and prioritizes masked positions that depend most on already-unmasked tokens, without additional training (Zhou et al., 2026). DEMASK introduces a dependency predictor to estimate conditional influences between masked positions, constraining parallel demasking (Ringel et al., 2026). These methods use structural information to determine which tokens can be predicted together and in what order. However, their primary objective is to organize generation rather than identify and repair errors in an already-generated reasoning trace.

Other approaches use higher-level structure to guide decoding. Planned Diffusion first generates a plan using an autoregressive model and then partitions the response into semantically independent chunks for parallel denoising (Israel et al., 2026). LogicDiff uses a lightweight role-prediction head to guide LLaDA through a predefined logical order, but its fixed template limits the extent to which the decoding order can adapt to the structure of an individual problem (Aman, 2026). C²DLM extracts a concept-level causal graph from a teacher model and uses it to guide attention during training (Han et al., 2025).

These methods use structure to decide when a token or sub-problem is generated. Error recovery must instead determine which already generated steps have become invalid, so a dependency representation used for scheduling does not, by itself, provide localized error correction.

2.4 Structured Reasoning and Verification

Chain-of-thought prompting (Wei et al., 2023) makes intermediate reasoning explicit, but this alone provides no mechanism for locating and correcting errors, and in autoregressive models correcting an earlier step generally requires regenerating its continuation.

Several methods use explicit structure to organize reasoning. Skeleton-of-Thought expands the components of an outline in parallel, reducing latency when they are independent, but this assumption is less suitable when later steps depend on earlier results (Ning et al., 2024). Plato organizes sub-problems into a dependency graph and decodes non-dependent nodes concurrently (Jin et al., 2025). Tree-of-Thoughts (Yao et al., 2023) and Graph-of-Thoughts (Besta et al., 2024) search over partial thoughts with branching, aggregation, and backtracking. In each case, the structure organizes prompting or search around a sequential decoder.

Verification offers a complementary way to find errors. Self-consistency aggregates the answers of several sampled chains without identifying the step responsible for a wrong one (Wang et al., 2023). Process supervision models score individual steps and so localize errors, but with an AR generator, the scores are mostly used to rerank or reject candidates (Lightman et al., 2023). Self-Refine revises an entire answer from model-generated feedback (Madaan et al., 2023), and Huang et al. (2024) show that intrinsic self-correction without reliable external feedback does not consistently improve reasoning and can reduce it.

In summary, token-level correction does not account for dependencies between reasoning steps, answer-level voting does not repair the reasoning behind an answer, dependency-aware decoding uses structure only to organize generation, and structured reasoning and verification methods provide no mechanism for localized remasking in diffusion models.

We address this gap with a graph-guided diffusion framework for inference-time correction in frozen masked DLMs. It represents the intermediate reasoning as a dependency DAG, which guides the denoising order and determines which steps must be reconsidered: verifiers identify inconsistent steps, and only the affected subgraph is remasked and regenerated, preserving independent reasoning paths without modifying the model parameters.

3 Methodology

3.1 Graph Construction

We convert each generated solution into a directed acyclic graph $G = (V, E)$. V contains a problem node P and one node per reasoning step. An edge $i \rightarrow j$ means that step j uses a quantity that step

i established. The parser is rule based and does not employ any learned components, which keeps the operation lightweight and deterministic. Every step node keeps its character span in the generation, which can be mapped to the token positions that the scheduler (§3.4) and the correction stage (§3.3) operate on. Figure 1 shows the full output for one GSM8K problem.

Nodes. The problem node P holds the values stated in the problem. Every other node is one reasoning step. We split the answer into steps at line breaks, or into sentences when it is written as a single paragraph. Each step node stores its character span, the values it uses and the values it produces.

Steps are of three kinds. A *header* is a line ending in a colon or a Markdown heading, such as “Cost of Shorts:”. Its numbers are ignored and it gets no edges. A *calculation* step contains at least one equation. We read every equals sign as a claimed equality between the expressions on either side, so a chain $a = b = c$ gives two equations that can each be checked on their own. Its single-number equation results are the values it produces, and its other numbers are the values it uses. A *claim* step has no equation, and every number it states counts as both used and produced. So “One pair of pants costs \$22.50” repeats a value from the problem and also makes it available to later steps.

Each value keeps its unit, read from up to three words after the number. “3 pairs of pants” has the head noun *pant* inside the container *pair*, and the edge rules compare head nouns. Two units are compatible if their head nouns match, if one head noun matches the other unit’s container (“12 cups” can supply “12 cups of feed”), or if either value has no unit.

Edges. An edge $i \rightarrow j$ means that step j uses a value that node i produced. Edges come from three rules, from strongest to weakest, and each edge records the rule that created it and its evidence: the values for value matching, otherwise the matched phrase or name.

Value matching. For every value that step j uses, we search backwards for the most recent step $i < j$ that produced an equal value with a compatible unit. “3 pairs of shorts” therefore cannot supply “3 pairs of pants”. A percentage can match either reading (40% as 0.4 or as 40). If no earlier step produced the value but the problem states it, the edge starts at P ; units are not checked against the

gsm8k_00026

Wrong: model 242, answer 243

arithmetic check: 2 false equations

Mishka bought 3 pairs of shorts, 3 pairs of pants, and 3 pairs of shoes. One pair of shorts costs \$16.50. One pair of pants costs \$22.50 and one pair of shoes costs \$42. How many dollars did Mishka spend on all the clothing items?

ANSWER SPLIT INTO STEPS

Step	Kind	Text
1	claim	To determine the total amount Mishka spent on all the clothing items, we need to calculate the cost of each type of clothing item and then sum these costs.
2	header	Cost of Shorts:
3	claim	Mishka bought 3 pairs of shorts.
4	claim	One pair of shorts costs \$16.50.
5	calc	Total cost for shorts = $3 * \$16.50 = \49 $\times 3 * \\$16.50 = \\$49 \rightarrow \text{actually } 49.5$
6	header	Cost of Pants:
7	claim	Mishka bought 3 pairs of pants.
8	claim	One pair of pants costs \$22.50.
9	calc	Total cost for pants = $3 * \$22.50 = \67 $\times 3 * \\$22.50 = \\$67 \rightarrow \text{actually } 67.5$
10	header	Cost of Shoes:
11	claim	Mishka bought 3 pairs of shoes.
12	claim	One pair of shoes costs \$42.
13	calc	Total cost for shoes = $3 * \$42 = \126
14	header	Now, we sum the costs of all the clothing items:
15	calc	Total cost = $\$49 + \$67 + \$126 = \242
16	claim	Therefore, Mishka spent a total of \$242 on all the clothing items.

DEPENDENCY GRAPH

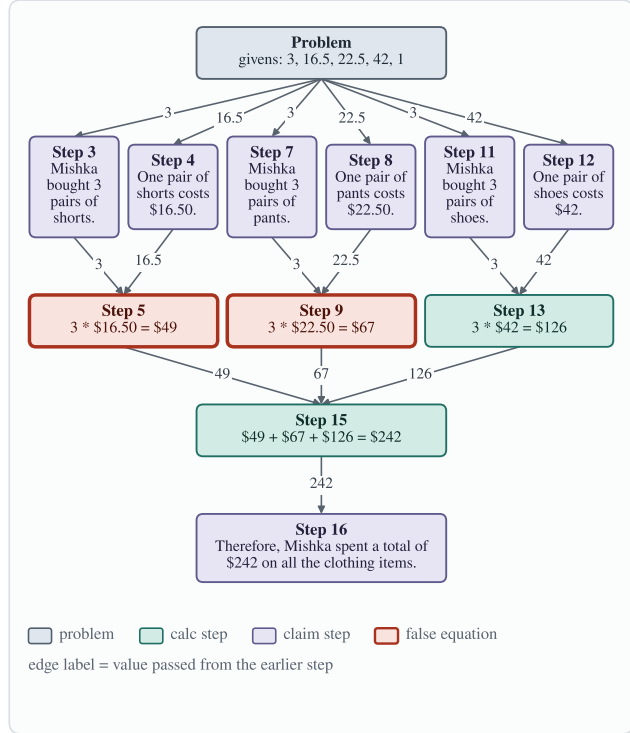


Figure 1: Parser and verifier output for GSM8K test problem 26. Left: the LLaDA answer split into steps, with headers in grey italics and the steps that failed verification highlighted. Right: the dependency graph, with edges labelled by the values they carry. Steps 5 and 9 contain wrong multiplications. Step 15 adds its inputs correctly but uses both wrong subtotals, so the answer is 242 instead of 243.

problem. When the same value occurs more than once in an equation’s expression (“ $180 + 180 + 15$ ”), each occurrence can link to a different earlier step, because the two terms are different quantities.

Explicit references. “Step 3” links to the step numbered 3 in the answer’s own list, and “from above” or “previous” to the step just before.

Shared names. A step that still has no incoming edge links to the closest earlier step that shares a capitalised name or a unit noun with it. This rule applies only as a fallback.

Edges only point from earlier steps to later ones, so G is acyclic by construction.

3.2 Verification

Training-free verifiers, which never see the gold answer, mark each step *invalid*, *valid* (checked and correct, which stops the propagation of $A(B)$ in §3.3) or *unverifiable*.

Arithmetic. For GSM8K and SVAMP, each equation on a graph node is recomputed with SymPy (Meurer et al., 2017) over exact rationals, and a false equation makes the step invalid. A softer *grounding* check flags an operand that neither the problem nor an earlier step supplies, ignoring common constants.

Logic. We parse the templated ProofWriter theories exactly and compute every provable literal under open-world semantics. A step, split at its causal cue (*since*, *because*, *so*) into reasons and a conclusion, is invalid if either is unprovable, if it cites a rule absent from the theory, or if it calls a provable literal unknown.

NLI. DeBERTa-v3-large (He et al., 2023) trained on NLI data including MNLI, ANLI and WANLI (Williams et al., 2018; Nie et al., 2020; Liu et al., 2022) checks each stated reason and cited rule against the relevant context. Off-the-shelf NLI does not apply rules (“The mouse is big. If the

Verifier	Data	Result
Arith.	Gold	0.9% of correct steps flagged; 92.3% of wrong results and 99.0% of swapped operators flagged
+Ground.	Gold	2.7% of correct steps flagged; 91.3% of copied operands flagged; first flag on the injected step in 91.4% of graphs
Arith.	LLaDA	50 of 86 wrong answers flagged at precision 0.94 (56 at 0.89 with grounding)
Logic	Gold	P = R = 1.00; 20,346/20,346 test labels
Logic	LLaDA	5.9% of steps flagged; 36 of 40 audited flags are genuine errors
NLI	Gold	Macro-F1 0.81; F1 0.91 at 14.5% FPR (threshold tuned on the other half)
NLI	LLaDA	Recall 0.72, precision 0.14 against the logic verifier

Table 1: Verifier results. Gold: 1,319 GSM8K test solutions as graphs (1,301 with one injected error); 501 ProofWriter depth-3 proof steps and 501 corrupted copies. LLaDA: GSM8K problems 1–250 (4-shot, one 256-token block, 65.6% accuracy) and the ProofWriter outputs of Table 2 (600 for logic, 200 for NLI).

mouse is big then the mouse is nice.” is judged neutral for “The mouse is nice”), so a rule bound to the step’s entity licenses the conclusion if its consequent entails it and the known facts entail each antecedent. Scores are calibrated by temperature scaling (Guo et al., 2017).

Results. Copied operands are caught mainly by grounding (12.2% without it; Table 1). Most other flags fall on graph descendants of the injected error (0.34 per graph) rather than on unrelated steps (0.05). The 30 unflagged wrong LLaDA answers contain no checkable false equation, in line with §4. The NLI verifier over-flags model-written steps, so it only backs up the logic verifier on steps that cannot be parsed. Temperature scaling ($T=3.2$) lowers its three-class ECE from 0.12 to 0.09, but binary ECE rises from 0.12 to 0.16.

3.3 Localized Remasking

Let B be the set of all steps marked as incorrect by the verifier. We define $A(B)$ as the affected subgraph, that is, the incorrect steps together with every step that depends on them. A step is added to $A(B)$ if it has a directed edge from any step already in $A(B)$, so the incorrect steps are propagated forward along the graph G . Propagation stops at a step that can be verified independently of B , in line with the idea that an inconsistency can be attributed to a minimal set of conflicting statements (Hunter and Konieczny, 2008). All steps outside $A(B)$ are

kept unchanged. In Figure 1, the verifier marks steps 5 and 9 as incorrect. $A(B)$ also contains step 15, whose own equation is correct but which uses both wrong subtotals, and step 16, which states the answer, while the independent step 13 is kept.

Each step s is linked to its token positions in the response through its token range $\sigma(s)$. The set of positions to be corrected is therefore

$$M = \bigcup_{s \in A(B)} \sigma(s). \quad (1)$$

We replace every position in M with [MASK] and re-denoise only these positions using the frozen DLM and its native decoding procedure, while all remaining positions stay fixed as context. This follows the remask-then-repredict principle of token-to-mask refinement (Yao, 2026). The difference is that the positions to remask are chosen by the reasoning graph and not by token confidence.

After each round, the trace is re-segmented, the graph is rebuilt, and the verifier is applied again. The procedure ends when no step is marked as incorrect or when a maximum number of rounds is reached. The number of denoising steps per round and the maximum number of rounds are fixed on the pilot runs. We measure the cost of a correction by the number of forward passes (NFE) and by the remask ratio $|M|/L$, where L is the response length.

To isolate the value of localization, we compare against two controls. Full-sequence remasking uses the same verifier and the same number of rounds, but remasks the entire reasoning trace whenever B is non-empty. A perfect-localization oracle sets B to the step known to be wrong, using a hand-labelled subset, and gives an upper bound on what the graph can achieve.

3.4 Dependency-Aware Scheduling

The scheduling step uses the graph to order denoising rather than to correct it. A node’s tokens are unmasked only after the tokens of its parents have been committed, and independent nodes are unmasked in the same denoising step. Because the graph is built from a written reasoning trace, the scheduled method first decodes a draft with the model’s native schedule, builds the graph from it, and then re-denoises the response in dependency order. The forward passes of the draft are included in the reported NFE. This protocol is our current plan and may change after the pilot runs. We evaluate scheduling separately from correction so that

its effect on accuracy and NFE can be attributed to the schedule alone.

3.5 Models and Datasets

Models. We use LLaDA-8B-Instruct (Nie et al., 2025) and Dream-v0-Instruct-7B (Ye et al., 2025), both frozen and run in bfloat16.

Datasets. We use GSM8K (Cobbe et al., 2021) (test set, 1,319 problems) and SVAMP (Patel et al., 2021) (1,000 problems), where arithmetic can be verified deterministically. We also use ProofWriter (Tafjord et al., 2021) in the open-world setting, with the depth-3 split sampled to 600 questions balanced across True, False and Unknown with a fixed seed. Each ProofWriter question carries the depth of its proof, and Unknown questions have no proof. ProofWriter tests the logic and NLI verifiers (§3.2).

3.6 Baselines and Protocol

We compare against vanilla decoding, Temporal Self-Consistency Voting, which aggregates the answers present at each denoising step without extra forward passes, DAPD, which schedules tokens using attention-based dependencies, and two trained token-level correctors, RemeDi-RL and ProSeCo, both run on LLaDA only. Each method runs with its official code and settings. All baseline runs are zero-shot with the chat template and 256 generated tokens, with prompts that ask for the answer after `The answer is`. LLaDA decodes greedily with low-confidence remasking in blocks of 32, and Dream uses entropy ordering with temperature 0.2 and top-p 0.95 and a fixed seed per example.

3.7 Evaluation Metrics

Metrics for all runs. *Accuracy* is the fraction of problems for which the extracted final answer matches the gold answer. The extractor first looks for the answer following `The answer is`, falls back to the last number or label in the output if this pattern is absent, and counts outputs containing neither as incorrect. We compare each method with the vanilla baseline on every dataset using a *paired bootstrap* 95% confidence interval for the accuracy difference and an exact *McNemar test*, which considers only the problems on which the two methods disagree and asks whether the wins are split evenly between them. We also report the number of problems *fixed* (incorrect under vanilla but correct under the evaluated method) and *bro-*

ken (correct under vanilla but incorrect under the evaluated method).

Inference cost is the average number of model calls per problem, reported as the number of forward passes (*NFE*) and including any corrector passes. *Output diagnostics* are the proportion of outputs that reach the token limit, the proportion that do not follow the requested answer format, and accuracy among outputs that state an answer. On ProofWriter, we additionally report accuracy by *gold label* and by *proof depth*. For *error analysis*, we report the proportion of incorrect GSM8K and SVAMP answers that contain an arithmetic error, found by recomputing every equation.

Metrics for the proposed method. The metrics above also apply to the proposed method. For NFE, we include the initial draft and all correction rounds, and we also measure inference time per problem on an otherwise idle GPU. We additionally evaluate four dimensions. *Reasoning consistency* is the proportion of traces with no detected inconsistencies between dependent steps, reported before and after correction. *Correction behavior* is characterized by the remask ratio, $|M|/L$, the fraction of response tokens remasked in each correction round and cumulatively across rounds, and by the affected-subgraph ratio, $|A(B)|/|V|$, the fraction of graph nodes affected by a detected inconsistency. *Correction quality* is the number of fixed and broken problems relative to the uncorrected output of the same model, which controls for sampling variability, and the accuracy gap relative to full-sequence remasking and the perfect-localization oracle. *Verifier quality* is the proportion of traces that can be parsed into a graph and the precision and recall of flagged steps on gold reasoning with injected errors (§3.2) and on a hand-labelled subset.

4 Baseline Results

We ran five methods under one shared protocol (zero-shot prompts, chat templates, 256 generated tokens, 256 denoising steps, and one scorer). Table 2 reports accuracy.

Four results shape the method. Choosing among the model’s own answers does not repair reasoning, since Temporal Voting changes math accuracy by under one percent. Token-level correction needs a trained corrector, because ProSeCo’s corrector applied to plain LLaDA-Instruct lowers GSM8K accuracy from 80% to 9% on a 100-example check.

Model	Method	Accuracy (%)		
		GSM8K	SVAMP	ProofWriter
LLaDA	Vanilla	80.6	88.4	67.5
	Temporal Voting	79.9	88.3	67.7
	DAPD	79.2	86.7	71.3 [†]
	RemeDi-RL [‡]	74.8*	93.3*	58.8*
	ProSeCo [‡]	74.1*	87.3	53.7*
Dream	Vanilla	80.4	85.4	44.3
	Temporal Voting	79.8*	85.4	56.8*
	DAPD	63.8*	77.7*	62.0*

Table 2: Accuracy (%) with $n = 1319, 1000$ and 600 .

*McNemar $p < 0.05$ against vanilla; [†] $p = 0.058$.

[‡]Trained correctors whose official evaluations use 1,024 tokens. At 256 tokens, 9–13% of their GSM8K outputs and 14–18% of their ProofWriter outputs reach the token limit, against none for vanilla LLaDA, and 16% (GSM8K) and 21–25% (ProofWriter) lack the requested answer format, against 5% and 0.3% for vanilla LLaDA.

Token-dependency scheduling is fast, using $3\text{--}10\times$ fewer forward passes, but it breaks dependencies between computed quantities, so DAPD loses 16.5 accuracy points on Dream’s GSM8K.

Trained correctors are strong when they finish but sensitive to budget. RemeDi-RL is the best method on SVAMP (+4.9). On GSM8K and ProofWriter, both trained correctors lose accuracy at 256 tokens, because many of their outputs reach the token limit or omit the requested answer format. Among outputs that do state an answer, RemeDi-RL is more accurate than vanilla (GSM8K 86.5% vs. 81.1%, ProofWriter 74.0% vs. 67.4%), whereas ProSeCo is on par (82.4% on GSM8K, 64.4% on ProofWriter). Because these comparisons are conditioned on the answered subset, we treat them as suggestive. RemeDi evaluations use 1,024 tokens (Huang et al., 2025), so we keep the shared 256-token table and will add a 512-token sensitivity run.

The large Dream gains on ProofWriter come from recovering unanswered questions. Vanilla Dream states no answer in the requested format for 168 of 600 questions, and accuracy among answered questions is about 62% for vanilla, voting, and DAPD (61.6%, 62.9%, and 62.0%).

Error analysis. We checked every calculation in the vanilla GSM8K and SVAMP outputs of both models. Only 16–27% of incorrect answers contain an arithmetic mistake detectable by a local check, and correcting all such slips would raise accuracy by at most 2.3–3.6%. In 58–75% of incor-

rect answers, every checkable calculation is correct, so the error lies in how quantities are chosen and combined across steps. On ProofWriter, LLaDA’s accuracy on True and False questions falls from 95% to 78%, 76%, and 60% at proof depths 0 to 3. Unknown questions are the main failure (LLaDA 47%, Dream under 3%), and 70 of the 106 LLaDA errors on Unknown questions are answered False.

5 Progress and Timeline

Completed work. We ran the baselines (vanilla decoding, Temporal Voting, DAPD, RemeDi-RL and ProSeCo, with the last two on LLaDA only) on three datasets and analysed where vanilla outputs fail. For the method, we implemented a rule-based parser that converts answers into dependency graphs with token spans (Figure 1) and the verifiers of §3.2. The remaining work integrates these components into the remasking loop, develops the scheduling strategy, and evaluates the resulting method.

Dates	Milestone
Oct 3–8	Integrate parser and verifiers into decoding.
Oct 9–15	Localized remasking and scheduling on LLaDA and Dream; tune on 100-problem pilots per dataset.
Oct 16–22	Full runs on three datasets and two models against the baselines and the scheduled variant.
Oct 23–26	Ablations, oracle controls, 512-token and RemeDi 1,024-token checks, timing and significance tests.
Oct 27–29	Final write-up, README and code cleanup; freeze the project on Oct 29.
Oct 30–31	Final revisions and submission.

References

- Shaik Aman. 2026. [LogicDiff: Logic-guided denoising improves zero-shot reasoning in masked diffusion language models](#). *Preprint*, arXiv:2603.26771.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2023. [Structured denoising diffusion models in discrete state-spaces](#). *Preprint*, arXiv:2107.03006.

- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. [Graph of thoughts: Solving elaborate problems with large language models](#). *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](#). *Preprint*, arXiv:2110.14168.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. [On calibration of modern neural networks](#). *Preprint*, arXiv:1706.04599.
- Kairong Han, Nuanqiao Shan, Ziyu Zhao, Zijing Hu, Xinpeng Dong, Junjian Ye, Lujia Pan, Fei Wu, and Kun Kuang. 2025. [C²DLM: Causal concept-guided diffusion large language models](#). *Preprint*, arXiv:2511.22146.
- Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. [DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing](#). *Preprint*, arXiv:2111.09543.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. [Large language models cannot self-correct reasoning yet](#). *Preprint*, arXiv:2310.01798.
- Zemin Huang, Yuhang Wang, Zhiyang Chen, and Guojun Qi. 2025. [Don’t settle too early: Self-reflective remasking for diffusion language models](#). *Preprint*, arXiv:2509.23653.
- Anthony Hunter and Sébastien Konieczny. 2008. Measuring inconsistency through minimal inconsistent sets. In *Proceedings of the Eleventh International Conference on Principles of Knowledge Representation and Reasoning*, KR’08, page 358–366. AAAI Press.
- Daniel Israel, Tian Jin, Ellie Cheng, Guy Van den Broeck, Aditya Grover, Suvinay Subramanian, and Michael Carbin. 2026. [Planned diffusion](#). *Preprint*, arXiv:2510.18087.
- Shuowei Jin, Xueshen Liu, Yongji Wu, Haizhong Zheng, Qingzhao Zhang, Atul Prakash, Matthew Lentz, Danyang Zhuo, Feng Qian, and Z. Morley Mao. 2025. [Plato: Plan to efficiently decode for large language model inference](#). *Preprint*, arXiv:2402.12280.
- Bumjun Kim, Dongjae Jeon, Moongyu Jeon, and Albert No. 2026. [DAPD: Dependency-aware parallel decoding via attention for diffusion LLMs](#). *Preprint*, arXiv:2603.12996.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. [Let’s verify step by step](#). *Preprint*, arXiv:2305.20050.
- Alisa Liu, Swabha Swayamdipta, Noah A. Smith, and Yejin Choi. 2022. [WANLI: Worker and AI collaboration for natural language inference dataset creation](#). *Preprint*, arXiv:2201.05955.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). *Preprint*, arXiv:2303.17651.
- Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, et al. 2017. [SymPy: symbolic computing in Python](#). *PeerJ Computer Science*, 3:e103.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. [Large language diffusion models](#). *Preprint*, arXiv:2502.09992.
- Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. 2020. [Adversarial NLI: A new benchmark for natural language understanding](#). *Preprint*, arXiv:1910.14599.
- Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. 2024. [Skeleton-of-Thought: Prompting LLMs for efficient parallel generation](#). *Preprint*, arXiv:2307.15337.
- Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. [Are NLP models really able to solve simple math word problems?](#) *Preprint*, arXiv:2103.07191.
- Liran Ringel, Ameen Ali, and Yaniv Romano. 2026. [Dependency-guided parallel decoding in discrete diffusion language models](#). *Preprint*, arXiv:2604.02560.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. [Simple and effective masked diffusion language models](#). *Preprint*, arXiv:2406.07524.
- Yair Schiff, Omer Belhasin, Roy Uziel, Guanghan Wang, Marianne Arriola, Gilad Turok, Ran Zilberstein, Michael Elad, and Volodymyr Kuleshov. 2026. [Learn from your mistakes: Self-correcting masked diffusion models](#). *Preprint*, arXiv:2602.11590.
- Oyvind Tafjord, Bhavana Dalvi Mishra, and Peter Clark. 2021. [ProofWriter: Generating implications, proofs, and abductive statements over natural language](#). *Preprint*, arXiv:2012.13048.

- Wen Wang, Bozhen Fang, Chenchen Jing, Yongliang Shen, Yangyi Shen, Qiuyu Wang, Hao Ouyang, Hao Chen, and Chunhua Shen. 2025. [Time is a feature: Exploiting temporal dynamics in diffusion language models](#). *Preprint*, arXiv:2508.09138.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). *Preprint*, arXiv:2203.11171.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. [Chain-of-thought prompting elicits reasoning in large language models](#). *Preprint*, arXiv:2201.11903.
- Adina Williams, Nikita Nangia, and Samuel R. Bowman. 2018. [A broad-coverage challenge corpus for sentence understanding through inference](#). *Preprint*, arXiv:1704.05426.
- Lin Yao. 2026. [Remask, don't replace: Token-to-mask refinement in diffusion language models](#). *Preprint*, arXiv:2604.18738.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. [Tree of thoughts: Deliberate problem solving with large language models](#). *Preprint*, arXiv:2305.10601.
- Jiacheng Ye, Shansan Gong, Liheng Chen, Lin Zheng, Jiahui Gao, Han Shi, Chuan Wu, Xin Jiang, Zhenguo Li, Wei Bi, and Lingpeng Kong. 2024. [Diffusion of thoughts: Chain-of-thought reasoning in diffusion language models](#). *Preprint*, arXiv:2402.07754.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. [Dream 7B: Diffusion large language models](#). *Preprint*, arXiv:2508.15487.
- Longxuan Yu, Yu Fu, Shaorong Zhang, Hui Liu, Mukund Varma T, Greg Ver Steeg, and Yue Dong. 2026. [Thinking out of order: When output order stops reflecting reasoning order in diffusion language models](#). *Preprint*, arXiv:2601.22035.
- Siyao Zhao, Devansh Gupta, Qinqing Zheng, and Aditya Grover. 2025. [d1: Scaling reasoning in diffusion large language models via reinforcement learning](#). *Preprint*, arXiv:2504.12216.
- Xueyu Zhou, Yangrong Hu, and Jian Huang. 2026. [DOS: Dependency-oriented sampler for masked diffusion language models](#). *Preprint*, arXiv:2603.15340.